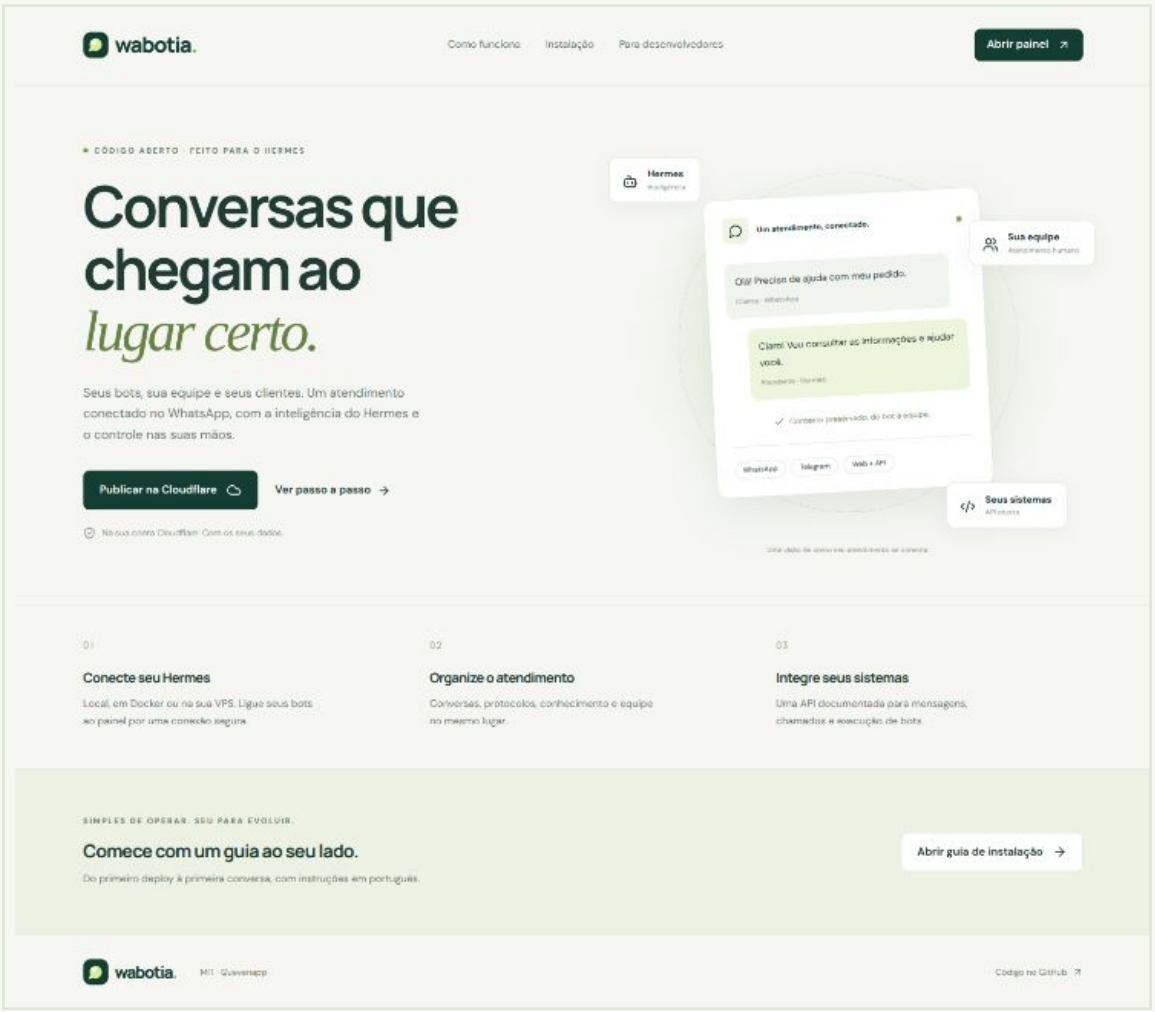
Seu atendimento. Bem conectado.

Guia para instalar na Cloudflare, conectar o Hermes e preparar sua equipe.



Página inicial do Wabotia. As ilustrações e conversas de exemplo são fictícias.

Começar em wabotia.queven.app/install

Edição de 05/11/2026. Leia na ordem; escolha apenas um modo de WhatsApp por número. O painel fica na Cloudflare. Hermes e QR continuam no computador ou servidor que você escolher.

Um instalador. Sua Cloudflare.

Você não precisa criar uma conta GitHub nem um repositório para instalar. O código aberto continua no repositório oficial. Sua instalação recebe versões prontas e pode ser atualizada pelo painel.

1. Instale Node.js e npm no computador ou na VPS de onde vai publicar. O roteiro completo, incluindo libatomic1 no Ubuntu, está nas páginas 3 a 6.

2. Prepare a conta Cloudflare: Workers, R2 e D1. Para conectar um Hermes privado, ative também Zero Trust. Veja páginas 7 a 9.

3. Crie um token da API Cloudflare e copie seu Account ID. O token autoriza o instalador a criar os recursos na sua conta. Página 10.

4. Execute o instalador no computador uma vez. Ele baixa o pacote oficial, verifica a integridade, publica os recursos e gera as chaves. Página 11.

5. Guarde a pasta privada criada. Abra a URL exibida e use SETUP_TOKEN para criar seu administrador. Páginas 12 e 13.

6. Conecte o Hermes e os canais. Nas próximas versões, use Sistema e atualizações, páginas 37 e 38.

Vai reinstalar? Use um nome novo, como wabotia-atendimento. A instalação nova começa vazia e não importa o histórico antigo. Mantenha a anterior até conferir o novo painel. O roteiro de transição está na página 39.

O computador usado para instalar pode ser desligado depois. Hermes, QR e cloudflared precisam permanecer ligados no local onde executam os bots e canais.

Node.js: prepare o Ubuntu

Node.js executa o instalador; npm vem junto. Você pode publicar a partir do computador ou da VPS. O Wabotia fica na Cloudflare. Se usar a VPS, a pasta privada com as chaves ficará nela.

1. Abra o terminal certo

1. Hostinger: abra **VPS**, selecione o servidor e procure **Terminal** ou **Terminal do navegador**. Use o Ubuntu da VPS, fora do contêiner Docker do Hermes.

2. Ubuntu local: abra **Terminal**. Windows/WSL2: abra o aplicativo **Ubuntu** no menu Iniciar. Para Windows sem WSL ou macOS, use a página 6.

3. Digite o comando abaixo e pressione Enter. Ele informa qual usuário está executando os comandos.

```
whoami
```

2. Instale os pacotes necessários

Se a resposta foi **root**, como costuma acontecer na VPS Hostinger:

```
apt-get update
apt-get install -y ca-certificates curl git libatomic1
```

Se apareceu **outro nome**, use esta alternativa com sudo:

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl git libatomic1
```

Use somente a alternativa do seu usuário. **libatomic1** evita o erro libatomic.so.1. A senha de sudo não aparece enquanto você digita. Se houver erro, corrija antes de seguir.

Cole uma linha de cada vez. Espere o nome do usuário reaparecer antes da próxima. Copie apenas os comandos, sem root@servidor:~#. Não use apt-get no macOS.

Instale Node.js e confira

3. Instale o NVM e a versão LTS

No mesmo terminal Ubuntu e com o mesmo usuário da página anterior, execute uma linha de cada vez. NVM gerencia as versões do Node.js. O primeiro comando usa o instalador oficial. O download pode levar alguns minutos.

```
curl -fsSL https://raw.githubusercontent.com/nvm-sh/nvm/v0.41.0/install.sh | bash
source "$HOME/.nvm/nvm.sh"
nvm install --lts
nvm alias default 'lts/*'
nvm use --lts
```

LTS é a versão com suporte prolongado. Copie a linha source inteira, incluindo as duas aspas. Se o NVM disser que já está instalado, siga com as linhas seguintes. Não use sudo antes de nvm ou node.

4. Veja os dois resultados

```
node -v
npm -v
```

Exemplo de saída, não são comandos para copiar:

```
v26.13.0
11.20.0
```

Os números podem mudar. O Wabotia exige **Node.js 22.13 ou superior** e npm. Os dois comandos precisam mostrar as versões sem erro. Se houver erro, use a próxima página.

> Node.js instalado por NVM fica disponível para este usuário. Continue o instalador Wabotia no **mesmo terminal e usuário**, sem sudo. Instalar como root não instala automaticamente para o usuário hermes.

Com os dois resultados corretos, prepare a Cloudflare nas páginas 7 a 10 e execute o Wabotia na página 11.

Abrir o passo a passo com botões de copiar · NVM oficial

Node.js: resolva os erros

libatomic.so.1: cannot open shared object file

O Node.js foi instalado, mas falta uma biblioteca do Ubuntu. Repita a preparação da página 3 conforme seu usuário. Se já fez apt-get update, o comando de correção como root é:

```
apt-get install -y libatomic1
node -v
npm -v
```

Com usuário comum, coloque sudo somente antes de apt-get. Não precisa apagar o Node.js nem o Wabotia.

nvm: command not found ou node: command not found

Entre no Ubuntu com o mesmo usuário usado para instalar e execute:

```
source "$HOME/.nvm/nvm.sh"
nvm use --lts
node -v
npm -v
```

Se o arquivo nvm.sh não existir, volte à etapa 3 da página 4. Se estiver no Windows ou Mac, conclua o instalador gráfico e abra uma nova janela de terminal.

O terminal mostra somente >

Geralmente faltou fechar uma aspa. Pressione **Ctrl+C** e copie novamente a linha inteira. A linha source precisa de uma aspa antes de $HOME e outra depois de nvm.sh.

Falta de permissão ou bloqueio do npm.ps1

No Ubuntu, confira **whoami**: usuário comum precisa de sudo para apt-get; root não precisa. Se sua conta não puder usar sudo, use uma conta administradora. No PowerShell, confira o npm com **npm.cmd -v**.

Não avance enquanto node e npm não responderem com números de versão. O erro de biblioteca acontece antes da instalação do Wabotia; não exige apagar recursos da Cloudflare.

Node.js no Windows e Mac

Windows sem WSL2: instalador gráfico

1. Abra nodejs.org/en/download. Escolha **LTS** e **Windows**. Baixe **Windows Installer (.msi)**.

2. Em um PC Intel/AMD comum, selecione x64. Use ARM64 somente se o seu computador for ARM. Não copie os comandos Linux/NVM da página.

3. Abra o arquivo baixado, avance pelo assistente e mantenha **npm** e **PATH** nas opções padrão. Conclua.

4. Feche terminais antigos e abra uma nova janela de **PowerShell** pelo menu Iniciar. Confira:

```
node -v
npm.cmd -v
```

Se usa Ubuntu/WSL2 para executar o instalador, siga as páginas 3 e 4 dentro do aplicativo Ubuntu. Instalar no Windows não substitui essa preparação.

macOS: instalador gráfico

5. No mesmo site oficial, escolha **LTS**, **macOS** e baixe **macOS Installer (.pkg)** para o seu Mac.

6. Abra o arquivo e conclua o assistente com as opções padrão. Ele instala Node.js e npm.

7. Feche terminais antigos. Abra **Aplicativos > Utilitários > Terminal** e confira:

```
node -v
npm -v
```

Espere versões sem erro, por exemplo v26.13.0 e 11.20.0. Node.js deve ser 22.13 ou superior. Depois siga os requisitos Cloudflare e os comandos do seu sistema na página 11. Se gravar, pause antes de preencher tokens ou abrir arquivos de chaves.

Para que serve a Cloudflare?

Ela mantém o Wabotia disponível na internet e guarda seus dados. Cada serviço cuida de uma parte. O Hermes continua executando os bots na máquina ou VPS escolhida.

Serviço	Função no seu atendimento
Workers	Executa o painel web, a API e as regras do atendimento. O painel continua acessível com seu computador desligado.
D1	Banco de dados: guarda usuários, configurações, conversas, protocolos e registros.
R2	Armazena documentos, imagens, áudios e anexos. Um bucket é a área onde esses arquivos ficam guardados. Use bucket privado.
Durable Objects e alarmes	Organizam o processamento por perfil e retomam tarefas pendentes, mantendo a ordem e evitando operações duplicadas.
Zero Trust	Área da Cloudflare para administrar a proteção do acesso ao Hermes. Exige ativar um plano, como Zero Trust Free.
Access	Confere se a chamada ao Hermes vem do Wabotia autorizado. Usa aplicações, políticas Service Auth e um par Client ID/Secret.
Tunnel e cloudflared	Ligam a Cloudflare à máquina do Hermes sem abrir portas no roteador. cloudflared é o programa instalado nessa máquina.
DNS	Associa nomes como api-hermes.seudominio.com ao túnel correto. O domínio precisa estar ativo na Cloudflare.
API da Cloudflare	Recebe os arquivos do instalador e publica as atualizações solicitadas no painel, sem repositório próprio ou Builds.
Workers Secrets	Guardam MASTER_KEY e SETUP_TOKEN no ambiente protegido do Worker. Não publique esses valores no código.

Exemplo: a mensagem chega ao Worker; o registro vai para o D1 e o anexo para o R2. Durable Objects organizam o trabalho. Para executar um bot, o Worker passa por Access + Tunnel até o Hermes.

O plano do site/domínio, Workers, a assinatura R2 e Zero Trust são configurações distintas. Ativar um deles não ativa automaticamente os demais. D1 e Durable Objects seguem o plano Workers.

Ative R2 e confira D1

R2: adesão necessária, mesmo com franquia gratuita

1. Na conta Cloudflare da instalação, abra **Armazenamento e bancos de dados > R2 > Visão geral**.

2. Se aparecer a tela de adesão, conclua a ativação da assinatura R2. Preencha os dados de cobrança solicitados e confira termos e valores antes de confirmar. A franquia gratuita não dispensa essa adesão.

3. Confirme que aparece a lista de buckets ou **Criar bucket**. O instalador cria o bucket privado com o nome escolhido seguido de -media. Não é preciso criar um bucket antes.

R2 inclui uma franquia mensal gratuita e pode cobrar pelo uso excedente. Confira as condições atuais; gratuito não significa armazenamento ilimitado.

D1: banco incluído no plano Workers

4. Abra **Armazenamento e bancos de dados > Banco de dados SQL D1**. Confira a lista e o botão **Criar banco de dados**.

5. O D1 já faz parte de **Workers Free e Paid**. Não há uma assinatura D1 separada equivalente à adesão do R2. Limites e eventuais cobranças seguem o plano Workers.

6. O instalador cria o banco exclusivo. Não é necessário criar um banco de teste antes. Se já instalou, confira **Worker > Bindings > DB** e preserve o banco existente.

Confira o plano Workers

Workers Paid é a referência de produção do Wabotia. Confira o plano e as condições na área de assinatura do Workers. Zero Trust Free e o plano do domínio não alteram essa assinatura.

O deploy cria as ligações **DB** e **MEDIA** entre Worker, D1 e R2. Você não precisa gerar chaves D1 ou credenciais S3/R2 para preencher no Wabotia. Durable Objects e alarmes também são configurados pelo deploy.

Ativação oficial do R2 · Preços R2 · Planos D1

Zero Trust e requisitos finais

1. Para conectar Hermes privado, abra **Zero Trust > Controles de acesso > Aplicativos**. Se aparecer a apresentação, clique **Começar** e conclua a ativação do plano. Zero Trust Free atende esta configuração inicial.

2. Confira termos e eventuais informações de cobrança, mesmo escolhendo gratuito. Se pedir nome de equipe, escolha um disponível.

3. Confirme acesso a **Aplicativos** e **Credenciais de serviço**. Se já ativou, não repita a adesão. O Service Token, as políticas Access e o túnel serão criados depois, nas páginas 19 a 23.

Antes de executar o instalador

Requisito	Como saber que está pronto
Node.js e npm	Siga páginas 3 a 6. Node.js 22.13+ e npm respondem no terminal que será usado para publicar.
Conta Cloudflare	Você tem acesso à conta de instalação e copiou o Account ID.
R2 ativado	Lista de buckets ou Criar bucket disponível.
D1 e Workers	D1 acessível e plano Workers escolhido. Produção usa Paid como referência.
Zero Trust e domínio	Aplicativos e Credenciais de serviço acessíveis. Domínio ativo para o túnel Hermes.

As duas chaves são geradas automaticamente

O instalador gera **MASTER_KEY** e **SETUP_TOKEN**, guarda ambas no Worker e no arquivo privado **chaves.local.json**. Você não precisa usar o gerador do site nesse caminho. MASTER_KEY protege as credenciais; SETUP_TOKEN libera o primeiro administrador.

O token Cloudflare da próxima página é outra credencial: ele autoriza criar e atualizar recursos. Guarde a pasta privada do instalador. Não use a senha de login como chave.

Crie o token da instalação

1. Abra a Cloudflare e selecione a conta da instalação. Copie seu **Account ID** na visão geral da conta. Ele também aparece como código de 32 caracteres na URL do dashboard, depois de dash.cloudflare.com/. Não use Zone ID.

2. Clique no ícone do usuário e abra **My Profile > API Tokens > Create Token**. Em **Custom Token**, clique **Get started**.

3. Dê um nome, como Wabotia instalação. Adicione as três permissões abaixo, todas do tipo **Account**.

Permissão	Nível
Workers Scripts	Edit
D1	Edit
Workers R2 Storage	Edit

4. Em **Account Resources**, escolha **Include > Specific account** e somente a conta onde vai instalar. Defina a validade e confira o resumo.

5. Crie o token e copie o valor exibido uma vez. Guarde-o em um gerenciador de senhas até digitá-lo no instalador.

Não use Global API Key. O token será usado no seu computador e guardado como secret no seu Worker. Não é enviado ao site oficial. Para futuras atualizações, você pode trocá-lo por um token mais restrito, página 38.

O token para preparar Access, Tunnel e DNS do Hermes é separado e usa as permissões da página 34.

Execute o instalador

Conclua o roteiro Node.js das páginas 3 a 6. Use o mesmo terminal e usuário em que node e npm responderam. Na VPS Ubuntu, execute fora do contêiner Hermes. Cole uma linha de cada vez e pressione Enter.

Linux, macOS ou WSL2

```
curl -fL https://wabotia.queven.app/install-wabotia.mjs -o install-wabotia.mjs
node install-wabotia.mjs
```

Windows / PowerShell

```
Invoke-WebRequest https://wabotia.queven.app/install-wabotia.mjs -OutFile install-wabotia.mjs
node .\install-wabotia.mjs
```

1. Ao ver **AGUARDANDO VOCÊ**, informe seu **Account ID** e pressione Enter. Depois cole o **token de API Cloudflare**. Por segurança, ele não aparece enquanto você digita ou cola. Pressione Enter.

2. Escolha um nome novo, em letras minúsculas e hífens, como **wabotia-atendimento**. Não reutilize o nome de uma instalação existente.

3. A publicação pode levar alguns minutos. O instalador mostra a etapa e, a cada 10 segundos, o tempo aguardando resultado. Ao ver **Pronto**, abra o endereço com **/app**.

> O instalador roda uma vez. Ele não mantém um servidor no computador. A publicação e as próximas atualizações usam a API Cloudflare; não precisa vincular GitHub ou configurar Deploy Hook.

Abrir o site para copiar os comandos



No site, escolha seu sistema e copie os comandos. As chaves são geradas pelo instalador.

Guarde a pasta e suas chaves

Na pasta de onde você executou o comando, procure a nova pasta com o nome da instalação. Exemplo: wabotia-atendimento. Ela é privada; não envie para repositórios nem para suporte.

Arquivo ou pasta	Para que serve
chaves.local.json	Guarda MASTER_KEY, SETUP_TOKEN e o token Cloudflare de atualização. Abra em um editor de texto e copie somente o valor desejado.
installation.local.json	Identifica a conta, o Worker e o progresso. Permite retomar uma instalação interrompida.
wrangler.jsonc	Configuração de recuperação com os recursos da instalação.
worker, web e migrations	Cópia da versão instalada e migrações do banco. Não contêm um repositório Git.

1. Faça uma cópia protegida da pasta em um local de backup. Preserve MASTER_KEY: sem ela, as credenciais guardadas no banco não podem ser recuperadas.

2. Abra chaves.local.json. Copie o valor de SETUP_TOKEN, sem as aspas, para a primeira conta. Na página 13 você escolherá sua senha de login, que é diferente dessas chaves.

Se a instalação parar

Corrija o erro indicado e, na pasta onde está install-wabotia.mjs, retome com o mesmo nome e diretório:

```
node install-wabotia.mjs --name wabotia-atendimento --resume
```

Digite novamente a conta e um token válido. A retomada preserva as chaves existentes. Se escolheu outra pasta, acrescente --directory com o caminho correto.

Sua primeira conta

Abra **SUA_URL/app**. A primeira conta é de administrador. As próximas pessoas serão cadastradas dentro do painel.



Tela real de primeira instalação. Os campos foram mantidos vazios.

1. Preencha nome, e-mail e senha com pelo menos 12 caracteres. Cole o **SETUP_TOKEN** do arquivo chaves.local.json.

2. Clique **Criar conta e entrar**. Depois clique **Criar perfil de atendimento** ou no **+** ao lado do seletor de perfil. Dê um nome, como Atendimento da loja.

3. Mantenha o atendimento pausado. Em **Pessoas**, crie uma conta por atendente e selecione os perfis permitidos.

Hermes: por onde começar

Se o Wabotia já está instalado, mantenha as chaves originais. Agora você vai ligar as APIs do Hermes, proteger a conexão na Cloudflare e cadastrar o perfil no painel.

1. Escolha onde está o Hermes: **computador nativo** (páginas 15 e 16) ou **VPS Ubuntu com Docker** (páginas 17 e 18).

2. Teste as APIs localmente. Só continue depois dos dois resultados **OK**.

3. Você pode usar o **botão de deploy pelo painel, páginas 34 a 36**, ou configurar Zero Trust, Access e Tunnel manualmente nas páginas 19 a 23.

4. Cadastre a conexão no Wabotia, página 24. A página 25 explica como religar serviços.

Cada chave tem um lugar

Credencial	Quem gera / onde usar
MASTER_KEY e SETUP_TOKEN	Gerador/instalador Wabotia. Secrets do Worker, já guardados na instalação. Preserve os valores originais.
Token administrativo Hermes	Assistente prepare-hermes.py. Campo administrativo da conexão no Wabotia.
API_SERVER_KEY de wabotia	Assistente ou Hermes. Campo da chave do perfil na conexão.
Access Client ID e Client Secret	Cloudflare. Preencha o par nos campos Cloudflare Access da conexão.
Token do túnel	Cloudflare. O assistente o guarda criptografado para gerar o instalador privado do cloudflared. No modo manual, use apenas na máquina.

As chaves Hermes geradas ficam em **~/.config/wabotia/hermes/chaves.local.txt**. Abra em um editor, copie apenas os valores e guarde o arquivo privado. No Linux, Ctrl+H mostra pastas ocultas. Não publique chaves nem capturas que as revelem.

Comandos completos e referências oficiais

Local: habilite a API

Abra Terminal no Ubuntu/Linux/macOS, ou Ubuntu no Windows/WSL2, com o mesmo usuário que instalou o Hermes. Confirme que **hermes --help** funciona. Não use sudo nos comandos Hermes.

1. Se não tem uma cópia do código Wabotia, baixe-a. Se já tem, abra e atualize a pasta existente.

```
git clone https://github.com/Quevenapp/Wabotia.git
cd Wabotia
```

2. Execute a preparação abaixo. Troque **seudominio.com** pelo domínio ativo na sua Cloudflare. O perfil wabotia é criado somente se ainda não existir.

```
python3 scripts/prepare-hermes.py \
  --profile wabotia --create-profile \
  --admin-hostname hermes-local.seudominio.com \
  --api-hostname api-hermes-local.seudominio.com
```

3. Aguarde as quatro etapas do assistente. Ele faz backup privado, preserva chaves existentes e cria as que faltam.

4. Abra o perfil **wabotia** no Hermes e confira modelo, provedor e autenticação. Se necessário, use o seletor nativo:

```
hermes -p wabotia model
```

API desligada: a preparação configura gateway.api_server.enabled=true, host=127.0.0.1 e porta 8642. Criar a chave sozinho não inicia o processo. A próxima página liga os serviços.

O gateway é compartilhado entre perfis. A chave default, necessária para o listener, permanece no Hermes; o Wabotia recebe somente a chave do perfil escolhido. Canais e dados existentes são preservados. O assistente recusa perfis pausados e gateway.multiplex_profiles=false.

Local: ligue e teste

No Ubuntu/Linux com gateway instalado como serviço de usuário, cole uma linha de cada vez. A barra \ no fim da linha indica continuação do mesmo comando.

```
mkdir -p ~/.config/systemd/user
install -m 600 \
  ~/.config/wabotia/hermes/hermes-wabotia-admin.service \
  ~/.config/systemd/user/hermes-wabotia-admin.service
systemctl --user daemon-reload
systemctl --user enable --now hermes-wabotia-admin.service
systemctl --user restart hermes-gateway.service
```

Reinicie fora de um atendimento em andamento: o gateway pausa brevemente seus canais. Se aparecer **Unit not found**, use o gerenciador pelo qual seu Hermes já inicia. Não abra um segundo gateway. Sem gateway instalado, consulte **hermes gateway install --help**.

Confira o resultado antes de abrir o túnel

```
python3 scripts/prepare-hermes.py --check

OK · Descoberta: autenticado e recusando acesso sem chave.
OK · API de bots: autenticado e recusando acesso sem chave.
```

Esse teste confirma a descoberta, idempotência durável e autenticação. Não executa o modelo nem envia mensagens. Só prossiga quando as duas linhas estiverem OK.

Execução manual, Mac e persistência

No macOS ou sem systemd, mantenha este comando em outro terminal. Reinicie o gateway pelo método já usado:

```
python3 scripts/prepare-hermes.py --serve-admin
```

Fechar o terminal encerra esse backend. No Mac, use launchd para início automático. No Ubuntu, para manter os servicos de usuário após sair da sessão:

```
sudo loginctl enable-linger "$USER"
```

No Windows/WSL2, systemd precisa estar habilitado e o WSL deve permanecer ligado. Suspender ou desligar o computador interrompe os bots.

VPS Docker: prepare

Entre por SSH ou pelo terminal da Hostinger. Este roteiro usa a imagem oficial atual, usuário **hermes**, volume **/opt/data** e rede **host** no Ubuntu. Confira sua implantação antes de aplicar.

```
docker ps --format 'table {{.Names}}\t{{.Status}}'
docker inspect -f '{{.HostConfig.NetworkMode}}' hermes
```

O exemplo usa contêiner chamado hermes. Substitua esse nome em todos os comandos se o seu for diferente. O segundo resultado precisa ser **host**. Em rede bridge, adapte a rede privada conforme o manual completo; o localhost do host não é o contêiner.

1. Faça backup do volume e do Compose existentes. Não use docker compose down -v.

2. Na pasta do código Wabotia da VPS, copie e execute o assistente dentro do contêiner:

```
docker cp scripts/prepare-hermes.py \
  hermes:/tmp/wabotia-prepare-hermes.py
docker exec --user hermes hermes python3 \
  /tmp/wabotia-prepare-hermes.py \
  --output /opt/data/wabotia-connection \
  --profile wabotia --create-profile \
  --admin-hostname hermes-vps.seudominio.com \
  --api-hostname api-hermes-vps.seudominio.com
docker exec --user hermes hermes cp \
  /tmp/wabotia-prepare-hermes.py \
  /opt/data/wabotia-connection/prepare-hermes.py
```

3. Fora de um atendimento, reinicie o contêiner existente para carregar a API:

```
docker restart hermes
```

Confira modelo/provedor no perfil novo. Se a imagem não tiver o usuário, Python ou API compatível, revise a versão; não contorne as permissões executando tudo como root. A execução na sua VPS precisa ser validada.

VPS Docker: mantenha ligado

Na VPS, crie **/etc/systemd/system/hermes-wabotia-admin.service** com sudo nano. O serviço abaixo mantém só o backend administrativo dentro do contêiner existente, sem iniciar outro gateway.

```
[Unit]
Description=Hermes administrativo para Wabotia
After=docker.service
Requires=docker.service

[Service]
Type=simple
ExecStart=/usr/bin/docker exec --user hermes hermes python3 \
  /opt/data/wabotia-connection/prepare-hermes.py \
  --output /opt/data/wabotia-connection --serve-admin
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Confira **command -v docker** e ajuste o caminho se necessário. A porta 9119 precisa estar livre. Caso contrário, prepare com --admin-port 9120 e use 9120 também no túnel.

```
sudo systemctl daemon-reload
sudo systemctl enable --now hermes-wabotia-admin.service
docker exec --user hermes hermes python3 \
  /opt/data/wabotia-connection/prepare-hermes.py \
  --output /opt/data/wabotia-connection --check
```

Espere os dois OK da página 16. Copie as chaves para um arquivo privado no host com os comandos do manual completo. Dentro do contêiner elas estão em **/opt/data/wabotia-connection/chaves.local.txt**.

Instale cloudflared no host Ubuntu. Na rede host, ele alcança as APIs em 127.0.0.1. Use o túnel e os hostnames **da VPS**. Preserve o volume e o reinício automático do contêiner ao atualizar o Hermes.

Cloudflare: Zero Trust

Se ativou Zero Trust nos requisitos iniciais, confira o acesso a Aplicativos e avance à próxima página. A referência abaixo orienta quem ainda precisa concluir a adesão. Use a conta com seu domínio ativo.



Tela real de ativação do Zero Trust. Confira os valores e termos vigentes antes de concluir.

1. Se aparecer a tela de boas-vindas, clique **Começar**. Selecione o plano adequado. Zero Trust gratuito, quando disponível, atende esta conexão inicial.

2. Confira preço, eventuais dados de cobrança e termos antes de confirmar a ativação. O plano Zero Trust é separado do Workers Paid.

3. Se pedir nome de equipe, escolha um disponível, como minhaempresa-wabotia. O endereço nome.cloudflareaccess.com não é a URL do Hermes.

4. Volte a **Controles de acesso > Aplicativos**. Continue quando conseguir acessar a lista e adicionar aplicações.

A ativação precisa ser concluída pelo titular da conta quando houver aceite de termos ou cobrança. Ter um túnel já criado não significa que o Access esteja habilitado.

Access: proteja as duas URLs

1. Abra **Controles de acesso > Credenciais de serviço > Service Tokens**. Em telas antigas: Access > Service Auth > Service Tokens.

2. Clique **Criar token de serviço**. Nome: wabotia-hermes-local. Escolha validade e anote a data de renovação.

3. A Cloudflare gera **Client ID** e **Client Secret**. Guarde os dois agora; o segredo pode aparecer apenas uma vez.

4. Em **Aplicativos > Adicionar aplicativo > Self-hosted**, preencha conforme a referência abaixo.

Referência de preenchimento: aplicação administrativa

Campo	Preencher
Nome	Wabotia - Hermes administrativo local
Subdomínio / domínio	hermes-local / seudominio.com
Caminho do Access	Vazio: proteger o hostname inteiro
Nome da política	Somente Wabotia
Action / Ação	Service Auth
Include / Incluir	Service Token = wabotia-hermes-local

5. Salve a política e a aplicação. Não use Everyone ou Bypass.

6. Repita para **api-hermes-local.seudominio.com**, com o mesmo token. Na VPS, substitua local por vps nos nomes.

7. Confirme as duas aplicações e suas políticas antes de publicar as rotas do túnel.

Service Auth permite que o Worker se identifique sem uma pessoa no navegador. Uma política de login por e-mail não substitui essa autenticação. A tabela é um mapa dos campos, não uma captura de uma configuração já salva.

Tunnel: crie a ponte

1. Abra **Rede > Túneis > Criar túnel**. Em algumas contas: Zero Trust > Networks > Connectors > Cloudflare Tunnels.

2. Escolha **Cloudflared**, se solicitado. Use um nome novo: wabotia-hermes-local; para VPS, wabotia-hermes-vps.

3. Na tela de instalação, selecione o sistema da máquina do Hermes. No Ubuntu, escolha o pacote/arquitetura corretos e instale pelos comandos oficiais exibidos.

4. Confirme a instalação no terminal:

```
cloudflared --version
```

5. A tela fornece um comando para registrar o conector. Ele contém o **token do túnel**. Guarde esse token privado; não o cole nos campos do Wabotia.

6. Se não houver outro serviço cloudflared, o comando oficial de instalação de serviço pode ser usado. Se já houver, use o serviço separado da próxima página.

7. Aguarde o estado **Healthy / Íntegro**. Depois configure as rotas da página 23.

Referência: o que cada nome significa

Nome	Uso
wabotia-hermes-local	Túnel novo para o computador local.
hermes-local.seudominio.com	Endereço HTTPS para descoberta de perfis.
api-hermes-local.seudominio.com	Endereço HTTPS para a API dos bots.
UUID.cfargotunnel.com	Destino DNS criado pela Cloudflare para o túnel.

Não execute outro **cloudflared service install** por cima de um conector existente. O token do túnel e o par Client ID/Secret do Access são credenciais diferentes. Não abra portas de entrada no roteador ou na VPS.

Um serviço de túnel separado

Ubuntu/Linux/WSL2 com systemd: no Bash da máquina do Hermes, guarde o token sem exibi-lo no histórico. Cole **somente o token** quando solicitado. A entrada fica invisível; pressione Enter.

```
mkdir -p ~/.config/wabotia/tunnel ~/.config/systemd/user
chmod 700 ~/.config/wabotia/tunnel
read -rsp "Token do novo túnel: " WABOTIA_TUNNEL_TOKEN
printf "%s" "$WABOTIA_TUNNEL_TOKEN" > \
  ~/.config/wabotia/tunnel/token
unset WABOTIA_TUNNEL_TOKEN
chmod 600 ~/.config/wabotia/tunnel/token
command -v cloudflared
```

Crie **~/.config/systemd/user/cloudflared-wabotia.service** no editor. Ajuste /usr/bin/cloudflared para o caminho mostrado acima. %h representa a pasta do usuário.

```
[Unit]
Description=Cloudflare Tunnel exclusivo do Wabotia
After=network-online.target

[Service]
ExecStart=/usr/bin/cloudflared tunnel --no-autoupdate run \
  --token-file %h/.config/wabotia/tunnel/token
Restart=always
RestartSec=5
UMask=0077

[Install]
WantedBy=default.target

systemctl --user daemon-reload
systemctl --user enable --now cloudflared-wabotia.service
sudo loginctl enable-linger "$USER"
```

No Mac/Windows nativo, siga o serviço oficial para esse sistema, preservando conectores existentes. Para teste temporário, cloudflared tunnel run --token-file CAMINHO_PRIVADO_DO_TOKEN mantém o túnel enquanto o terminal estiver aberto.

Rotas públicas e DNS

No túnel, abra **Published application routes** (ou Public Hostnames) e adicione duas rotas. Use os mesmos hostnames já protegidos pelo Access. A tabela é referência de preenchimento.

Campo	Administrativo	Bots
Subdomínio	hermes-local	api-hermes-local
Domínio	seudominio.com	seudominio.com
Path no Tunnel	^/api/profiles$	^/p/wabotia/v1/.*$
Tipo	HTTP	HTTP
URL do serviço	127.0.0.1:9119	127.0.0.1:8642
HTTP Host Header	127.0.0.1:9119	Deixar padrão

1. Em **Additional application settings > HTTP Settings**, defina o HTTP Host Header administrativo. Isso permite a validação de Host do backend loopback; Access e Bearer continuam obrigatórios.

2. Salve as duas rotas. Na VPS, use hermes-vps e api-hermes-vps. Se mudou a porta administrativa, ajuste os dois campos da rota.

3. Em **Domínios > seu domínio > DNS > Registros**, confira os CNAMEs criados para UUID.cfargotunnel.com, com proxy ativo. Não substitua registros que já tenham outra função.

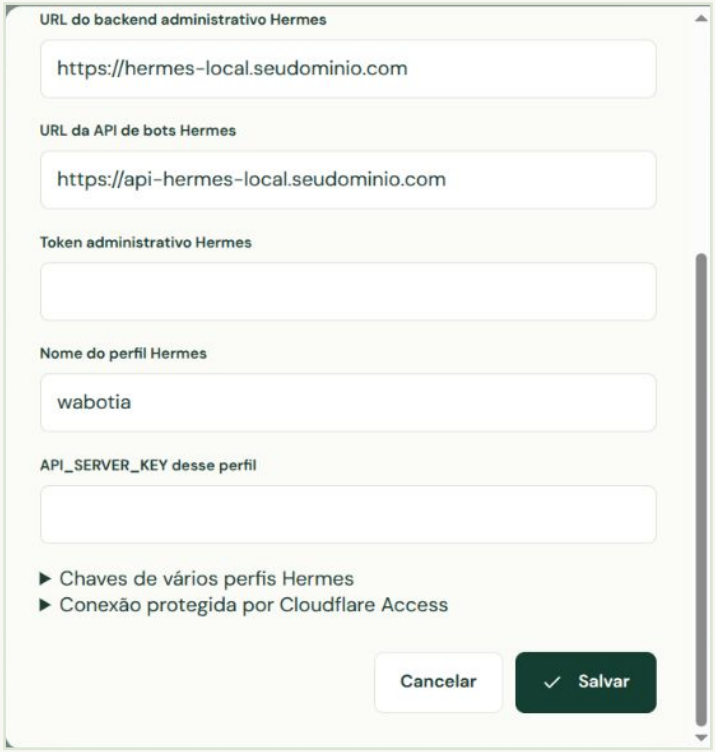
4. Abra uma rota protegida em janela anônima, sem chaves. O Access deve bloquear. Uma tela de login/recusa é esperada; a raiz / pode retornar 404.

5. Se aparecer JSON de perfis sem autenticação, remova a rota publicada e corrija a proteção antes de continuar. O teste autenticado será pelo Wabotia.

Dois campos diferentes: o caminho no Access ficou vazio para proteger o hostname inteiro. O Path do Tunnel limita as rotas publicadas. A API deste exemplo permite apenas o perfil wabotia. Não publique todas as páginas administrativas do Hermes.

Cadastre no Wabotia

Na URL da sua instalação, abra **/app**. Crie um perfil de atendimento e vá a **Conexões > Nova conexão > Hermes**.



Tela de referência do Wabotia, com dados fictícios e chaves vazias.

Campo	Preencher
URL administrativa	https://hermes-local.seudominio.com
URL dos bots	https://api-hermes-local.seudominio.com
Token administrativo	Token administrativo Hermes de chaves.local.txt
Perfil e API_SERVER_KEY	wabotia e sua chave de chaves.local.txt
Cloudflare Access	Abra os detalhes e cole Client ID + Client Secret

Use URLs base, sem /api/profiles ou /p/wabotia/v1. Salve, **Teste a conexão** e **Descubra bots**. Ative a conexão e escolha wabotia como Atendente em Equipe Bot. Mantenha o atendimento pausado até testar os canais.

Se desligou, como religar

Ubuntu nativo, com os serviços deste guia

```
systemctl --user start hermes-gateway.service
systemctl --user start hermes-wabotia-admin.service
systemctl --user start cloudflared-wabotia.service
python3 scripts/prepare-hermes.py --check
```

Se desativou a API na configuração, repita a preparação da página 15 e depois **systemctl --user restart hermes-gateway.service**. O assistente preserva as chaves existentes.

Docker na VPS

```
docker start hermes
sudo systemctl restart hermes-wabotia-admin.service
systemctl --user start cloudflared-wabotia.service
docker exec --user hermes hermes python3 \
  /opt/data/wabotia-connection/prepare-hermes.py \
  --output /opt/data/wabotia-connection --check
```

Se desativou a configuração da API, repita a preparação dentro do contêiner com o mesmo --output e reinicie hermes. Se usou o serviço cloudflared oficial, utilize seu nome correspondente.

Sintoma	Ação
Connection refused	Iniciar gateway/backend e conferir portas.
502 na Cloudflare	Túnel não alcança a API. Execute --check local.
1033 / túnel offline	Ligar cloudflared; conferir token e internet.
401/403 ou login HTML	Conferir chaves, validade e política Service Auth.
Invalid Host	HTTP Host Header = 127.0.0.1:9119.
Perfil não executa	Conferir modelo/provedor e chave do perfil.

Com dois OK locais e túnel Íntegro, volte ao Wabotia e teste a conexão. Computador/WSL precisam ficar ligados. O painel permanece disponível quando o Hermes está offline.

WhatsApp: Meta direto

Escolha este modo para receber a API oficial Meta diretamente no Worker. O Hermes é chamado pelas APIs nativas; não há plugin Wabotia nesse caminho.

1. Na Meta, prepare a aplicação WhatsApp Business e um número de teste. Separe o token, **Phone Number ID**, **WABA ID** e **App Secret**.

2. No Wabotia, abra **Conexões > Nova conexão** e escolha **WhatsApp · API oficial Meta**. Informe o número com DDI, somente dígitos, e os dados da etapa anterior.

3. Crie um **Verify Token** aleatório e preencha o campo. Salve a conexão.

4. Copie o webhook exibido no cartão da conexão. Na aplicação Meta, cadastre essa URL e o mesmo Verify Token. Assine os eventos **messages**.

5. Clique **Testar conexão**, ative o canal e use somente destinatários de teste autorizados na Meta.

6. Teste uma mensagem, a resposta pelo painel e um anexo. Só depois prepare o número de produção.

```
https://SUA_URL/webhooks/meta/ID_DA_CONEXAO
```

Janela, consentimento e templates

Fora da janela de atendimento de 24 horas, esta versão permite notificações de protocolo somente com consentimento e template **UTILITY aprovado**. O corpo deve ter **{{1}}** para protocolo e **{{2}}** para andamento. Configure nome e idioma em Menus e preferências.

> **Um número, um consumidor.** Antes de trocar entre Meta direto, QR ou Hermes, pause o antigo, pare o conector correspondente e confira as pendências em Entregas. Ative o novo modo somente depois da reconciliação.

A confirmação perdida de um envio pode produzir estado unknown. Confira no canal antes de autorizar nova tentativa: ela pode duplicar uma mensagem já recebida.

QR direto ou WhatsApp Hermes

QR direto: processo ao lado do Hermes

1. Crie uma conexão **WhatsApp · QR direto**. Informe o número. Salve e clique **Gerar chave do conector**. Guarde chave e ID da conexão.

2. No host que ficará ligado, abra **connectors/qr**. Copie .env.example para .env e preencha URL HTTPS, ID e chave. Instale e inicie:

```
cd connectors/qr
npm ci
npm start
```

3. Ative a conexão. O QR aparece no cartão do painel. No WhatsApp do celular, abra **Aparelhos conectados** e leia o código.

4. Mantenha o processo ligado. Preserve **data/**, com sessão e diário; configure reinício e backup. Não precisa abrir portas de entrada nem túnel para esse conector.

Em Docker, após preencher .env nessa pasta, use **docker compose up -d --build**. Preserve o volume qr-data ao atualizar. Baileys é um cliente não oficial; o pareamento deve ser validado na sua conta.

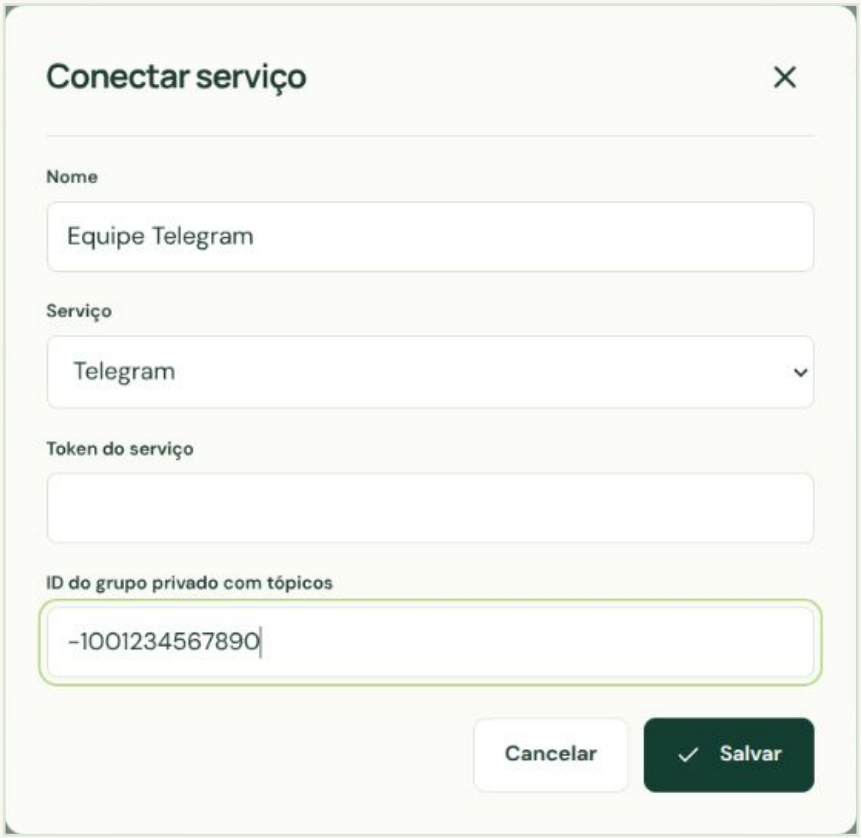
WhatsApp que já está ligado no Hermes

Mantenha Meta/QR configurados no Hermes e escolha **WhatsApp · pelo Hermes** no painel. Esse caminho precisa do transporte mínimo e da extensão compatível de entrada exclusiva no gateway.

Com o gateway parado e o perfil copiado para backup, siga o README de **connectors/hermes**: conferir a revisão, testar o patch com git apply --check, instalar no perfil existente e habilitar wabotia_transport em in_process. Reinicie pelo processo que já utiliza.

Não misture os caminhos. WhatsApp direto e Telegram direto dispensam esse plugin. A extensão do Hermes requer manutenção técnica e revalidação quando o gateway é atualizado; não aplique em outra revisão sem conferir compatibilidade.

Telegram para sua equipe



Nova conexão > Telegram. O grupo é fictício e o token não está preenchido.

1. Crie um bot exclusivo no **@BotFather** com /newbot. Não reutilize o bot do gateway Hermes.

2. Crie um **grupo privado com tópicos**. Adicione o bot como administrador, com permissão de gerenciar tópicos e enviar mensagens e anexos.

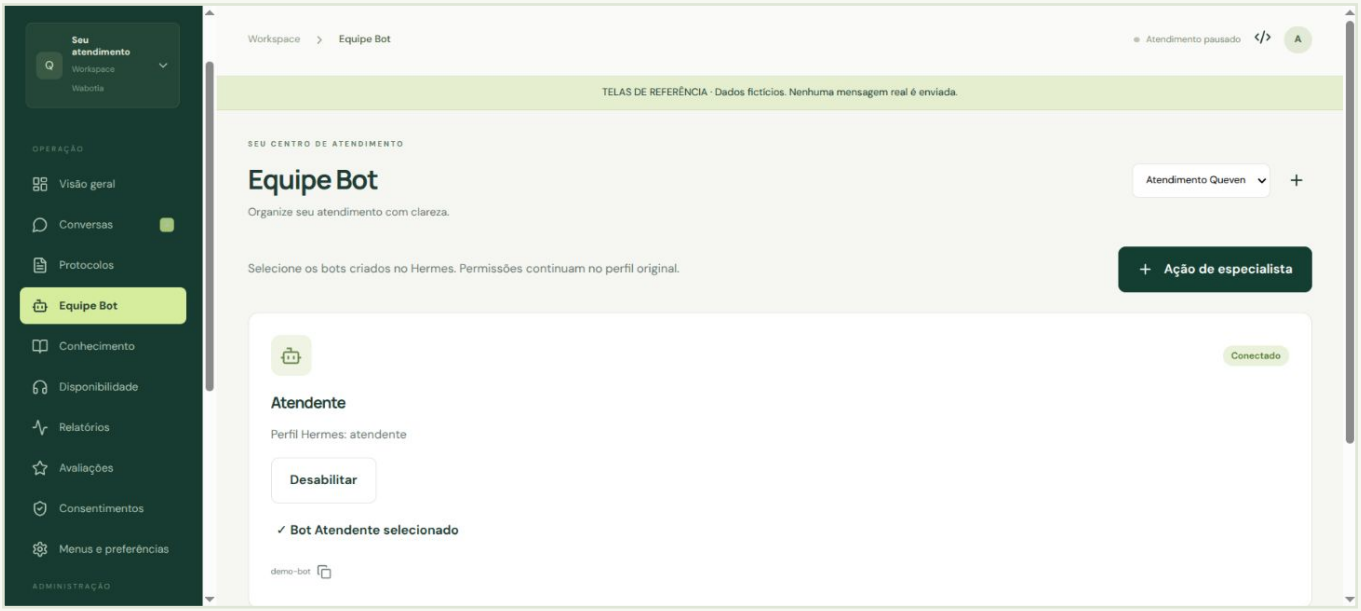
3. Obtenha o ID numérico do grupo, geralmente -100...; o Bot API getUpdates pode mostrar esse ID antes de configurar o webhook.

4. No painel, informe token e grupo, salve, teste e clique **Ligar recebimento Telegram**. Ative a conexão. Em Menus e preferências, cole os IDs da conexão e do grupo.

5. Cada pessoa envia **/identificar** no grupo. Vincule o ID retornado à conta correta em Pessoas. Depois use **/presente**.

O Telegram conversa diretamente com o Worker. Não instala plugin no Hermes. Vincule apenas o ID verificado da pessoa; não use bots terceiros para acessar os dados do grupo.

Prepare os bots e os menus



Equipe Bot com um perfil de demonstração. Os dados são fictícios.

1. Em **Equipe Bot**, habilite os perfis descobertos e selecione quem será o **Atendente**. Só habilite perfis cuja API_SERVER_KEY foi cadastrada.

2. Clique **Ação de especialista**. Escolha o bot, descreva quando consultá-lo e a ação permitida. Os modelos e ferramentas continuam configurados no Hermes.

3. Abra **Conhecimento** e cadastre informações úteis: texto, URL pública, PDF com texto ou DOCX. Espere a importação ficar disponível. Arquivos têm limite de 20 MB.

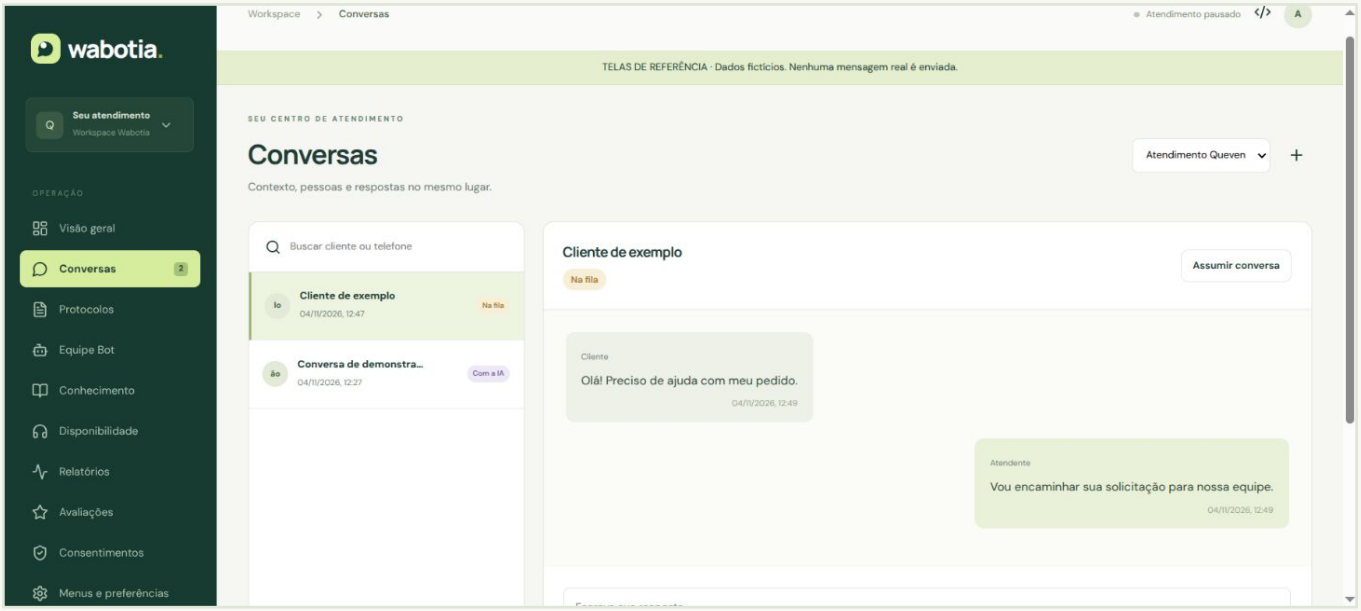
4. Em **Menus e preferências**, configure prazo, instruções, avaliações e opções de menu. Se necessário, use o editor JSON para aplicativos e submenus. JEV é opcional.

5. Cadastre pessoas e permissões por perfil. Em **Disponibilidade**, registre presença antes de abrir o atendimento.

Arquivos de bots: precisam ser exportados como artefatos da API nativa. Caminhos locais não viram links públicos. Downloads de uso único interrompidos exigem nova exportação; o Wabotia não os repete automaticamente.

PDF digitalizado sem texto não tem OCR nesta versão. Vídeos precisam de legendas públicas/autorizadas; também é possível importar TXT, VTT ou SRT.

Faça a primeira conversa



Tela de atendimento com mensagens fictícias. Assumir invalida a resposta automática anterior.

1. Use um número de teste autorizado. Envie uma pergunta que possa ser respondida com o conhecimento cadastrado.

2. Abra a conversa e clique **Assumir**. Confira que o responsável é sua conta e responda pelo painel.

3. Teste nota interna, anexo, transferência e abertura de protocolo. Nota interna não deve ir ao cliente.

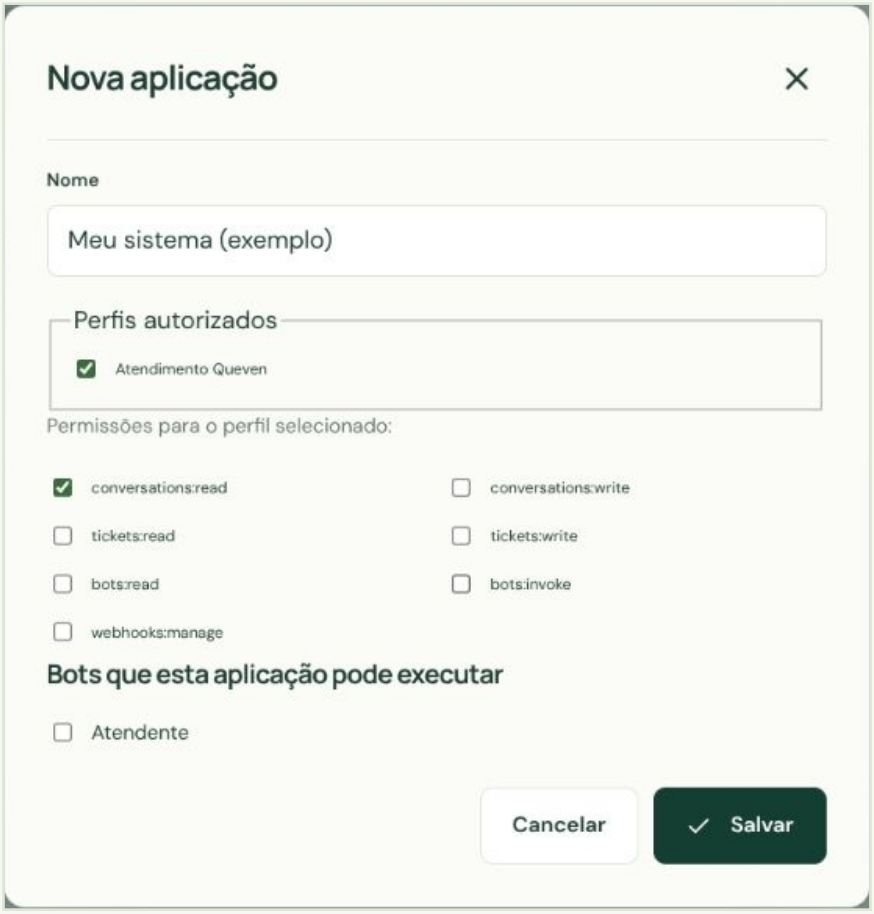
4. No Telegram, vá ao tópico da conversa e use /assumir antes de /responder. Confira o resultado também no painel.

Comando Telegram	Uso
/presente ou /ausente	Registra disponibilidade.
/assumir e /responder texto	Assume a conversa e responde ao cliente.
/nota texto e /transferir ID	Registra nota interna e transfere para atendente autorizado.
/encerrar e /retomar	Conclui ou devolve o atendimento à automação.
/identificar e /ajuda	Mostra sua identidade ou a lista de comandos.

No modo por comando, texto comum vira nota interna. No modo direto, o responsável responde sem /responder conforme as regras do tópico. Confira esse comportamento com mensagens fictícias antes de usar.

Integre outros sistemas

A API é para comunicação entre servidores. Seu programa usa uma chave própria e só opera os perfis, bots e ações autorizados.



API e integrações > Nova aplicação. Exemplo de seleção de permissões.

1. Cadastre uma aplicação, escolha os perfis e marque as permissões necessárias. Selecione os bots que ela poderá executar.

2. Clique **Gerar chave**, escolha validade e copie o valor exibido para o backend do integrador. Para rotação, emita outra e depois revogue a antiga.

3. Abra **/api-docs** na sua instalação. Há OpenAPI 3.1 e exemplos em cURL, JavaScript e Python no repositório.

4. Cadastre um webhook HTTPS para eventos. Guarde o segredo HMAC e verifique assinatura, timestamp e ID do evento. O destinatário precisa deduplicar.

Escritas exigem Idempotency-Key. Bots retornam 202 e um ID para acompanhamento. Repetir a chave com outro corpo gera conflito; timeout não deve disparar uma nova execução automaticamente.

Backup e manutenção

Copiar seus dados

1. Pause o atendimento e as entradas dos canais. Pare QR/transporte para não gravar durante a cópia. Preserve os diários.

2. Na pasta Wabotia, execute o comando. Use o arquivo de configuração da implantação correta.

```
node scripts/backup.mjs wrangler.production.local.json backup-seguro
```

O backup inclui banco e mídias. Só considere concluído quando manifest.json tiver **complete: true**. Guarde a pasta protegida e a **MASTER_KEY original separadamente**. Depois retome os canais.

Restaurar em recursos novos

Crie um D1 vazio e um R2 privado novo. Faça uma configuração local apontando para esses recursos, sem sobrescrever a produção. Então execute:

```
node scripts/restore.mjs wrangler.restore.local.json backup-seguro --empty-target
```

O script recusa banco preenchido. Configure a MASTER_KEY original, publique a mesma versão e teste dados e arquivos antes de mover o domínio. Pare a instalação antiga antes de ligar os mesmos canais na restaurada.

Diagnóstico

```
node scripts/diagnose.mjs https://SUA_URL
```

Sintoma	O que conferir
Hermes offline	Processo, portas, túnel, Access, URLs e chaves de cada perfil.
401 / 403 / 404	Validade da chave, permissões, perfil e dono do recurso.
429	Respeite Retry-After e reduza a frequência.
unknown	Confira Hermes/canal e use Execuções ou Entregas para reconciliar.

Antes de atender clientes

O núcleo e a publicação já foram testados. Seus perfis e canais precisam passar por homologação com credenciais e destinatários autorizados. Não confunda uma tela de demonstração com um canal conectado.

Execute na sua instalação de teste

1. Envie e receba texto e anexo em cada canal escolhido. Confira protocolo, prazo, consentimento, avaliações, presença, notas e transferência.

2. Peça uma resposta lenta ao bot e assuma no meio da execução. A resposta automática antiga não deve chegar ao cliente.

3. Desligue e religue Hermes, túnel e conector. O painel deve continuar acessível; confira a retomada e os estados sem confirmação.

4. Repita uma mensagem/webhook de teste. Confira deduplicação, assinatura e recuperação. Teste também uma chave expirada/revogada.

5. Valide backup e restauração antes de depender da instalação. Só então ative o atendimento para clientes.

Evidência desta edição: 50 testes em workerd, 3 testes do atualizador, 3 testes Python do transporte, 20 verificações HTTP em staging, exemplos em três linguagens e restauração remota D1/FTS/R2. Os testes externos usaram simulações ou dados fictícios. **As APIs Hermes locais também passaram na descoberta e na recusa de acesso sem chave. Isso não substitui homologação completa de modelo, túnel e canais Meta, QR e Telegram na sua instalação.**

Onde encontrar os detalhes

Site e guia atualizado

Código aberto e instruções completas

Matriz de validação e limites

Operação, atualização e recuperação

Uma organização por implantação. Fuso America/Sao_Paulo. O histórico antigo não é importado automaticamente. Telas capturadas em 04/11/2026; menus de fornecedores podem mudar. Wabotia é independente de Cloudflare, Meta, Telegram e NousResearch.

Autorize o deploy do Hermes

Depois de ligar as APIs locais e obter dois OK, você pode criar Access, túnel e DNS pelo Wabotia. O Hermes continua no computador ou VPS. Quem já conectou esta máquina pode manter a conexão existente.

1. Ative Zero Trust e confirme que o domínio está ativo na Cloudflare. As adesões a R2 e aos planos continuam sendo feitas pelo titular.

2. Abra **My Profile > API Tokens > Create Token > Custom Token**. Limite o token à conta e à zona do domínio desejado.

3. Na conta, conceda **Cloudflare Tunnel: Edit**, **Access: Apps and Policies: Edit** e **Access: Service Tokens: Edit**. Na zona, conceda **Zone: Read** e **DNS: Edit**. Defina validade e copie o token em local privado.

4. Na visão geral do domínio, copie **Account ID** e **Zone ID**.

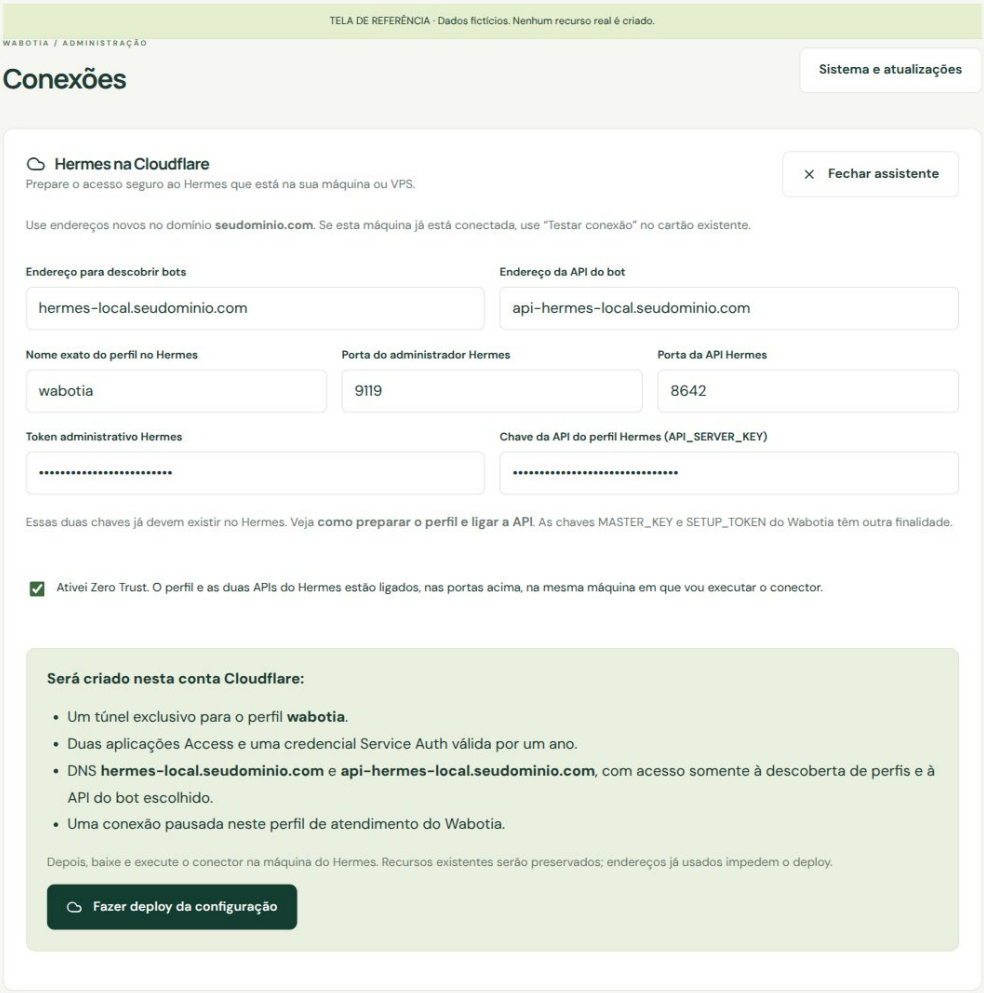
5. No Wabotia, abra **Sistema e atualizações > Preparação do Hermes na Cloudflare**. Cole os dois IDs e o token. Clique **Verificar e salvar autorização**.

Qual chave é esta? É o token de API da Cloudflare, usado para criar os recursos. Ele é diferente de MASTER_KEY/SETUP_TOKEN, das duas chaves Hermes e do par Client ID/Secret do Access. O Wabotia o guarda criptografado.

Se aparecer erro de autorização, confira conta, zona, validade e todas as permissões. Um token somente de leitura não pode executar o deploy.

Roteiro completo do deploy pelo painel

Revise e faça o deploy



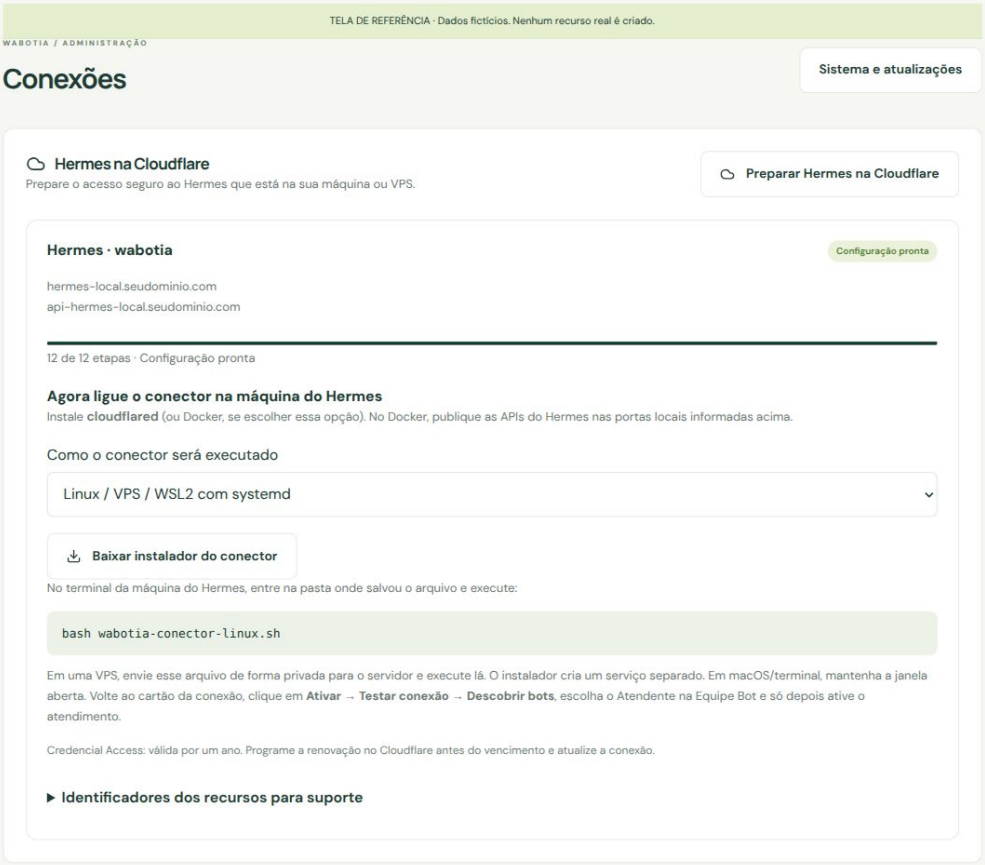
Conexões > Preparar Hermes na Cloudflare. Tela funcional com dados e chaves fictícios.

1. Informe dois subdomínios novos, perfil Hermes, portas e as duas chaves de chaves.local.txt. Confirme os pré-requisitos e clique **Revisar configuração**.

2. Confira o resumo e clique **Fazer deploy da configuração**. Acompanhe as 12 etapas. Endereços usados serão preservados e impedirão a criação.

A credencial Access gerada vale por um ano. Renove-a no Cloudflare e atualize a conexão antes de vencer. Não execute o assistente nos endereços de uma conexão já pronta.

Ligue o conector da máquina



Ao concluir o deploy, baixe o conector para a máquina do Hermes. Exemplo fictício.

1. Instale cloudflared pelo site oficial, ou Docker para essa opção. Escolha o sistema e baixe o instalador. O arquivo contém uma credencial: guarde-o em local privado.

2. No terminal da máquina do Hermes, entre na pasta do download e rode o comando correspondente. Na VPS, transfira o arquivo de forma privada antes.

```
bash wabotia-conector-linux.sh
```

Linux/WSL2 precisa de systemd. Docker requer Linux com rede do host. macOS/terminal precisa permanecer aberto. Na VPS, habilite sudo loginctl enable-linger "$USER". Depois, ative a conexão no painel, teste, descubra os bots e selecione o Atendente.

Atualize pelo painel



Sistema e atualizações. Tela de referência da versão 0.3, sem credenciais.

1. O instalador já habilita a atualização. Entre como administrador e abra **Sistema e atualizações**. Faça backup antes de publicar uma nova versão.

2. Clique **Verificar atualização**. O painel consulta as versões oficiais publicadas; um commit no GitHub só fica disponível quando seu pacote é publicado.

3. Se houver uma versão nova, clique **Atualizar Wabotia**. O painel acompanha a preparação dos arquivos, as migrações e a publicação.

4. Espere **Atualização publicada e verificada**. Clique **Reabrir painel atualizado**. Banco, mídias, domínios e credenciais são preservados.

Você não precisa sincronizar uma cópia GitHub, configurar Builds nem criar Deploy Hook. A consulta é renovada a cada cinco minutos.

Renove o token quando precisar

Se o token inicial expirar ou você quiser reduzir suas permissões, crie outro token na mesma conta Cloudflare. Depois da instalação, a atualização precisa somente de **Account > Workers Scripts > Edit**.

1. Na Cloudflare, abra **My Profile > API Tokens > Create Token > Custom Token**. Adicione Workers Scripts: Edit e restrinja à conta da instalação.

2. No Wabotia, abra **Sistema e atualizações > Alterar configuração de atualização**.

3. Cole o token no campo protegido e clique **Verificar e salvar token**. O Wabotia confirma se o Worker é desta instalação antes de guardar a credencial criptografada.

4. Depois de confirmar o novo token, revogue o anterior na Cloudflare se ele não for usado por outro serviço. Atualize também sua cópia privada de chaves.local.json.

Não troque MASTER_KEY para renovar o token. Essa chave protege credenciais já guardadas no banco. O token de atualização e o token de deploy da configuração Hermes têm finalidades diferentes.

Onde ver o andamento

Na Cloudflare, abra **Workers & Pages > seu Worker > Deployments / Implantações**. A versão do Wabotia também aparece em **SUA_URL/api/health**. As novas instalações publicam diretamente, sem uma compilação em Builds.

Documentação de atualização e recuperação

Reinstalação e recuperação

Sair de uma instalação antiga

1. Faça backup do banco, arquivos e credenciais da instalação antiga se precisar preservar o histórico. A nova instalação começa vazia.

2. Execute o instalador com **outro nome**. Crie seu administrador e cadastre a conexão com o Hermes. Reutilize suas URLs, chaves e túnel existentes quando forem adequados.

3. Mantenha os canais da instalação nova pausados durante a preparação. Antes de transferir um número ou bot, pause a instalação antiga e confira as entregas pendentes.

4. Atualize os webhooks dos canais para a URL da nova instalação. Teste atendimento humano, bots e anexos antes de ativar. Nunca use dois consumidores ativos para o mesmo número.

5. Só depois de validar a nova instalação considere remover o Worker e os recursos antigos ou a cópia particular do GitHub. A nova instalação não depende deles.

A atualização ficou sem confirmação

Confira Implantações e /api/health. Um pedido pode ter sido aceito antes de a conexão cair. Não repita enquanto houver publicação em andamento. Após falha confirmada ou 15 minutos sem confirmação, o painel permite liberar nova tentativa, exigindo conferir que a anterior terminou.

> Não restaure apenas o código se uma migração exigir outro esquema de banco. Use o backup e revise a compatibilidade. O instalador não substitui outro Worker nem troca as chaves de uma instalação existente.

O novo instalador foi exercitado em Linux com recursos reais na Cloudflare. Comandos Windows/macOS estão documentados; ainda precisam de validação nesses sistemas. Teste canais reais na sua instalação antes de usar em produção.